

PUESTA EN MARCHA

y verificación del hardware

Datalogger Ambiental IoT · YD-ESP32-S3

Hallazgos reales del primer montaje y solución de problemas

12 de agosto de 2026

www.acusticaescolar.com · Licencia MIT

Resumen

Este documento recoge los hallazgos de la primera puesta en marcha real del datalogger, verificados sobre el montaje físico. Complementa a la Guía de Montaje v2 y al Documento de Proyecto. Su objetivo es que cualquiera que replique el montaje pueda resolver los mismos problemas que aparecieron la primera vez, sin repetir el proceso de diagnóstico.

✓ Estado final verificado: 5/5 componentes operativos (SHT41, DBMETER, DS3231, Senseair S8 y LittleFS), registro de datos en flash funcionando, lectura de datos confirmada, y exportación a CSV compatible con Excel.

1. Configuración del Arduino IDE

Core y placa:

Core ESP32	esp32 de Espressif Systems, versión 3.2.0. Verificado con Arduino IDE 2.3.6.
Board	ESP32S3 Dev Module. NO elegir «ESP32-S3-USB-OTG»: es otra placa distinta, con configuración cerrada que no corresponde a este módulo.
Flash Size	16MB (128Mb). Debe coincidir con la flash real del módulo N16R8.
PSRAM	OPI PSRAM. El N16R8 lleva PSRAM octal; con «QSPI PSRAM» o «Disabled» puede haber cuelgues.
Flash Mode	QIO 80MHz
USB CDC On Boot	Enabled. Crítico en esta placa: con «Disabled» el monitor serie queda en blanco (ver 2.2).
Upload / Monitor	Upload 921600 · Monitor 115200 baudios

Resto de opciones del menú Herramientas (dejar en sus valores por defecto):

CPU Frequency: 240MHz (el firmware la baja a 80 MHz por software al arrancar) · Core Debug Level: None · USB DFU On Boot: Disabled · Erase All Flash Before Sketch Upload: Disabled · Events Run On: Core 1 · Arduino Runs On: Core 1 · JTAG Adapter: Disabled · USB Firmware MSC On Boot: Disabled · Upload Mode: UART0 / Hardware CDC · USB Mode: Hardware CDC and JTAG

Partition Scheme: capacidad de almacenamiento

Con el esquema por defecto («Default 4MB with spiiffs»), LittleFS dispone de unos 1.400 KB, suficientes para 11 días de registro. La placa tiene 16 MB de flash, de los que quedan unos 12 MB sin asignar. Para campanas mas largas puede elegirse un esquema de 16 MB, comprobando que la particion de datos sea del tipo que LittleFS monta (SPIFFS): los esquemas etiquetados FATFS no sirven sin adaptar el firmware. Verificar siempre la capacidad real con el comando `l` tras cambiar el esquema.

Librerías necesarias (Library Manager):

Sensirion I2C SHT4x	por Sensirion. Clase SensirionI2cSht4x. Constante SHT40_I2C_ADDR_44.
S8_UART	por Josep Comas (jcomas). El fichero de cabecera es s8_uart.h (minúsculas).
RTCLib	por Adafruit.
LittleFS	incluida en el core ESP32 — no se instala.

2. Hallazgos críticos de la puesta en marcha

Estos cinco puntos causaron fallos durante el primer montaje y son los primeros a verificar si algo no funciona.

2.1 Puente IN-OUT del pin 5V (causa de fallo del S8)

⚠ La placa YD-ESP32-S3 tiene un puente (jumper) IN-OUT que conecta el VBUS del USB al pin de 5V de los headers. VIENE ABIERTO DE FÁBRICA.

Con el puente abierto, el pin 5V da solo ~400 mV residuales en lugar de 5V. Consecuencia: el Senseair S8 no recibe alimentación suficiente, la lámpara NDIR no pulsa (no parpadea su LED rojo interno) y el CO₂ da 0 permanentemente, aunque el sensor responda por UART.

Síntoma característico:

- El S8 responde por UART (se lee su firmware) pero da CO₂ = 0 siempre.
- El LED rojo interno del S8 no parpadea (la lámpara NDIR no mide).
- Medido con multímetro: el pin 3V3 da 3,3V correctos, pero el pin 5V da ~400 mV.

✓ **Solución:** cerrar el puente IN-OUT. Entonces el pin 5V entrega los 5V del VBUS y el S8 mide correctamente (verificado: 485–596 ppm en aire de interior).

⚡ **Precaución:** con el puente cerrado, NO alimentar simultáneamente por el USB y por el pin 5V externo con dos fuentes distintas (conflicto de 5V). Como todo se alimenta desde el banco USB (fuente única), no hay problema.

2.2 USB CDC On Boot debe estar en Enabled

En esta placa, el Serial (monitor serie) sale por el USB nativo del ESP32-S3, no por el chip CH343P. Si USB CDC On Boot está en Disabled, no se ve nada en el monitor serie aunque el programa funcione perfectamente.

✓ **Configuración correcta:** USB CDC On Boot = Enabled. Síntoma si está mal: el LED hace su secuencia y el programa corre, pero el monitor serie queda en blanco.

2.3 Bug del core ESP32 3.2.0 — el UART se cuelga al arrancar

HardwareSerial.begin() con pines asignados se cuelga si la línea RX ya tiene tráfico en el arranque. El S8 empieza a transmitir en cuanto se alimenta, así que el programa se bloquea justo al iniciar el UART del S8. Afecta a los cores 3.1 a 3.3.

Solución aplicada en el firmware — arrancar el UART sin pines y asignarlos después:

```
S8_serial.begin(9600, SERIAL_8N1, -1, -1);
S8_serial.setPins(PIN_S8_RX, PIN_S8_TX);
```

2.4 Orientación del UART del S8 (confirmada)

La orientación que funciona, NO intercambiar:

ESP32 GPIO1 TX (col 45J)	→ S8 UART_RxD (col 2J)
ESP32 GPIO2 RX (col 46J)	← S8 UART_TxD (col 3J)

⚡ Test útil: si al intercambiar TX/RX el S8 no responde a nada (firmware vacío, IDs a 0), esa orientación es la incorrecta. Si el S8 SÍ responde el firmware pero da CO₂=0, el problema NO es el cruce UART — mirar la alimentación (puente IN-OUT, apartado 2.1).

2.5 Dispositivo I²C extra en 0x57 (normal)

El escaneo I²C detecta un cuarto dispositivo en 0x57 además de los tres esperados (0x44, 0x48, 0x68). Es la EEPROM AT24C32 integrada en el módulo DS3231 (modelo ZS-042). Es inofensiva, no interfiere, y queda disponible por si se quiere usar como almacenamiento adicional.

3. Pinout definitivo verificado

Protoboard apaisada, columnas 1–63 de izquierda a derecha, bloque inferior filas A–E (A abajo), bloque superior filas F–J (J arriba). ESP32 con USB-C a la derecha (col 63).

Señal	Col-Fila	Destino / notas
GPIO8 SDA	53 B	Bus I ² C — carril inferior
GPIO9 SCL	56 B	Bus I ² C — carril inferior
GPIO1 TX → S8	45 J	→ S8 RxD (col 2J)
GPIO2 RX ← S8	46 J	← S8 TxD (col 3J)
GPIO48 LED	57 J	WS2812B integrado
3V3	42 B	Alimentación sensores — carril superior
5V	62 B	Alimentación S8 (requiere puente IN-OUT cerrado)
GND	42 J	Masa común — carril superior

Senseair S8 (montado tumbado cruzando el canal, cols 1–5):

Pin S8	Col-Fila	Conexión
G+ (5V)	1 A	← ESP32 5V (col 62B)
G0 (GND)	2 A	→ carril GND
UART_RxD	2 J	← ESP32 GPIO1 TX (col 45J)
UART_TxD	3 J	→ ESP32 GPIO2 RX (col 46J)

Sensores I²C (todos en fila B, bloque inferior):

Sensor	Cols	Pinout
SHT41 (girado 180°)	12–15 B	12B=GND · 13B=VCC · 14B=SCL · 15B=SDA
DBMETER	20–24 B	20B=VCC · 21B=INT(nc) · 22B=SCL · 23B=SDA · 24B=GND
DS3231	29–34 B	29B=GND · 30B=VCC · 31B=SDA · 32B=SCL · 33B/34B=nc

4. Pull-ups I²C — opcionales con estos breakouts

Verificación con multímetro sobre el montaje real:

Con 2 resistencias externas de 4,7 k Ω	SDA-3V3 = 1,88 k Ω (innecesariamente bajo)
Sin resistencias externas	SDA-3V3 = 3,7 k Ω (valor ideal)

Los breakouts (DS3231 y/o DBMETER) ya incorporan pull-ups internos que dan 3,7 k Ω , el valor óptimo para I²C a 100 kHz ($\approx 0,89$ mA por línea).

✓ Decisión: NO instalar las resistencias externas de 4,7 k Ω con estos breakouts. El bus funciona correctamente solo con los pull-ups integrados.

⚡ Procedimiento de comprobación para otros breakouts: medir SDA-3V3 y SCL-3V3 con el multímetro (sistema sin alimentar). Si la lectura está entre $\sim 2,2$ y 4,7 k Ω , no añadir externas. Si es mucho mayor de 4,7 k Ω (bus débil), añadir una por línea. Pendiente: medir SCL-3V3 (debería dar $\sim 3,7$ k Ω como SDA).

5. Operación: comandos serie y volcado de datos

5.1 Comandos por el monitor serie

El firmware acepta comandos escritos en el monitor serie sin interrumpir el registro de datos:

T2026-08-06 06:50:00	Ajustar el reloj DS3231 (formato exacto TAAAA-MM-DD hh:mm:ss)
H	Mostrar la hora actual del RTC
D	Volcar todo el CSV guardado
I	Info del fichero y espacio en flash
L	Destello de comprobación: verifica que el equipo sigue operativo
X	Borrar el CSV (pide confirmación 'SI')
?	Ayuda con la lista de comandos

 **Ajuste de hora:** elige una hora redonda un poco por delante, escribe el comando T con esos segundos, y pulsa Enter justo cuando el reloj de referencia marque esa hora exacta. Así el ajuste queda preciso al segundo. El DS3231 la mantiene con su pila CR2032.

5.2 Recuperación de las lecturas guardadas (volcado con el .bat)

Los datos registrados en la flash del ESP32 se recuperan a un fichero CSV en el PC mediante dos ficheros que van juntos en la misma carpeta:

volcar_datos.bat	El lanzador. Es el que ejecutas con doble clic.
volcar_datos.ps1	El script de PowerShell que hace el trabajo. No se toca.

No requiere instalar nada (usa PowerShell, ya incluido en Windows).

Procedimiento paso a paso:

1. Guardar volcar_datos.bat y volcar_datos.ps1 en la misma carpeta del PC.
2. Conectar el ESP32 por USB (un solo cable) con el firmware corriendo.
3. CERRAR el Monitor Serie del Arduino IDE si está abierto.
4. Doble clic en volcar_datos.bat.
5. El script detecta el puerto, pide el volcado y guarda el CSV. Muestra cuántos registros ha guardado y la ruta del fichero.

Qué hace automáticamente:

- Detecta el puerto COM del ESP32, ignorando COM1 y COM2 (puertos fantasma del sistema) y buscando el dispositivo USB-serie real por su descripción.
- Envía el comando D al firmware y captura solo lo que hay entre las marcas '----- INICIO CSV -----' y '----- FIN CSV -----'.
- Filtra cualquier línea [LOG #n] que se cuele durante el volcado.
- Guarda el resultado en la subcarpeta 'datos_volcados', con nombre fechado: datalog_2026-08-06_071530.csv. No sobrescribe: cada volcado genera un fichero nuevo.

El CSV está preparado para Excel en español:

- Primera línea 'sep=,' para que Excel separe por columnas automáticamente al abrir con doble clic.
- Decimales con coma (29,75 en vez de 29.75), entrecomillados para no confundir con el separador de columnas.

⚠ El Monitor Serie del Arduino IDE debe estar CERRADO al ejecutar el volcado: solo un programa puede usar el puerto COM a la vez. Si aparece 'No se pudo abrir el puerto', esa es la causa: ciérralo y reintenta.

⚡ La primera vez, Windows puede mostrar un aviso de SmartScreen por ser un .bat. Elegir 'más información → ejecutar de todas formas'. El .ps1 se puede abrir con el Bloc de notas para comprobar que solo lee el puerto serie y guarda un fichero.

Para un .xlsx nativo (con formato o gráficos): abrir el CSV en Excel y usar 'Guardar como' → formato Excel. Es un paso manual, sin necesidad de instalar nada.

5.3 Ventana horaria y capacidad de almacenamiento

El firmware registra solo entre las 07:00 y las 19:00, todos los días incluidos fines de semana. Fuera de esa franja permanece en espera sin escribir. Esto reduce a la mitad el volumen de datos y garantiza la autonomía para estudios de una semana.

Con intervalo de 30 s y ventana de 12 h/día, la autonomía sobre los ~1.380 KB disponibles es:

Sin alertas	19,2 días
Alertas ocasionales	16,9 días
Peor caso (alertas continuas)	11,4 días

✓ Una semana completa necesita 847 KB incluso en el peor caso: queda un 39 % de margen. Para cambiar el horario, editar HORA_INICIO y HORA_FIN al principio del firmware.

5.4 Formato del CSV

Cada fila contiene: timestamp_iso8601, temp_C, hum_pct, co2_ppm, dB_LAeq, dB_max, alertas.

- dB_LAeq es el nivel continuo equivalente (promedia energía, no decibelios, que es la magnitud correcta en acústica) y dB_max el pico del intervalo. El sonómetro se muestrea cada 125 ms y su promediado interno se ajusta automáticamente a ese valor en el arranque: verifícalo en la fase 1, debe indicar «promediado interno ajustado a 125 ms».
- El campo de alertas lleva las etiquetas de las variables fuera de rango separadas por ';', u 'OK' si todo está correcto.
- Etiquetas de condición: T_BAJA, T_ALTA, T_RIESGO, HR_BAJA, HR_ALTA, CO2_DEFIC, CO2_MALA, CO2_RIESGO, DB_ELEV, DB_RIESGO.
- Etiquetas de fiabilidad: ERR_TH, ERR_CO2, ERR_DB (sensor sin lectura válida), ERR_RTC (marca de tiempo no fiable) y FLASH_BAJA (poco espacio libre). Si aparecen, esa variable debe excluirse del análisis en las filas afectadas.

✓ El firmware verifica cada escritura y avisa por consola si un registro no se pudo guardar, llevando la cuenta de los fallos. Los comandos I y L informan de ellos. Esto evita que un estudio desatendido pierda datos en silencio.

⚡ Testigo luminoso: verde tenue (3/255) mientras todo funciona; rojo intermitente y brillante si algo requiere atención (sensor sin respuesta, fallo de escritura o poco espacio libre). No señala condiciones ambientales, para no alterar la conducta de los ocupantes; una avería, en cambio, debe verse desde lejos para poder intervenir.

6. Protocolo de arranque verificado

Secuencia completa para poner en marcha una unidad desde cero:

6. Cerrar el puente IN-OUT de la placa (una sola vez, ver 2.1).
7. Configurar el Arduino IDE (ver sección 1), con USB CDC On Boot en Enabled.
8. Instalar las cuatro librerías.
9. Cargar el firmware `datalogger_test_v1.ino` por el puerto COM izquierdo.
10. Abrir el Monitor Serie a 115200 baud. Debe aparecer el arranque y las 5 fases.
11. Verificar 5/5 componentes OK en el resumen. Tras el diagnóstico el LED pasa a verde tenue: es el testigo con el que quedará durante el estudio. Si en algún momento parpadea en rojo, el equipo requiere atención.
12. Ajustar la hora con el comando T.
13. Limpiar los datos de prueba con el comando X antes del primer estudio real.
14. Para extraer datos: cerrar el Monitor y ejecutar `volcar_datos.bat`.

✓ Lecturas de referencia esperadas en interior: CO₂ 400–1200 ppm, temperatura ambiente, humedad 40–70%, sonido 34–45 dB SPL en ambiente tranquilo. El S8 tarda ~30 s en dar la primera lectura estable tras el encendido.

© www.acusticaescolar.com — Licencia MIT

Se concede permiso para usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y/o vender copias de este documento y del software asociado, con la condición de que se incluya el aviso de autoría en todas las copias. El contenido se proporciona sin garantía de ningún tipo.